Project: Dashboard to Rule the Academic World

Note: All updates, FAQ, and bug fixes, if any, will be announced in this post. Please check it frequently.

Overview

You will build an application that is presented through a "dashboard" interface at a website, which embeds usage of relational (MySQL), document (MongoDB), and graph (Neo4j) databases. Your application will allow target users to use and exploit the Academic World dataset, which we have used in our MPs, in some useful and interesting scenarios. You will submit your code, a video demo, and a report/README for your work.

Objectives

1. Create an application to explore and use the Academic World dataset.
2. Build an end-to-end application using databases.
3. Use different types of databases as the backend.
4. Design a dashboard as the frontend to use and monitor data.

Team Work

We encourage you to work in a **team of two**. You can also choose to work **individually**. In either case, the expectation of the amount of work and grading criteria for a team will be the **same**, and you should **register your team here by March 22**.

To find a teammate, you can enter yourself into our Looking for Teammates list. You can contact anyone on the list via direct messages on Campuswire. After you find a teammate, please remove yourself from the list.

You and your teammate should create a repository and form a team in the GitHub organization *CS411-Chang*. Please follow these instructions for setting up the Github repository.

Expected Time

We expect the project can be comfortably completed in totally 30-60 hours in a team. In our testing by three CAs, each took about 20 hours to individually complete a sample project.

Questions/Feedback

If you have any questions or if you need any feedback, you can ask your TA/CA. You can ask in your personal chat room on Campuswire. Or, you can create a work document in Google Doc, share it with your TA/CA, and tag them for questions, just like your MPs.

Deadline: Week 15 (May 03, 2026)

Submission

Submit the following deliverables to GitHub by the deadline.

a) Code Repo (**R1**)

Upload your code to our Github organization *CS411-Chang*. See instructions for setting up the Github repository.

b) Report/README (**R2**)

Write README.md in your repository to document your code. It should document your project work, consisting of the following:

- **Title:** Name of your application.
- **Purpose:** What is the application scenario? Who are the target users? What are the objectives?
- **Demo:** Give the link to your video demo. Read the video demo section below to understand what contents are expected in your demo.
- **Installation:** How to install the application? You don't need to include instructions on how to install and initially populate the databases if you only use the given dataset.
- **Usage:** How to use it?
- **Design:** What is the design of the application? Overall architecture and components.
- **Implementation:** How did you implement it? What frameworks and libraries or any tools have you used to realize the dashboard and functionalities?
- **Database Techniques:** What database techniques have you implemented? How?
- **Extra-Credit Capabilities:** What extra-credit capabilities have you developed if any?
- **Contributions:** How each member has contributed, in terms of 1) tasks done and 2) time spent?

There is no requirement for the length, but make sure it is detailed enough to follow what each team member has done.

c) Video Demo (**R3**)

Record a video to demo the application. The video demo should be somewhere in the range of 5 to 10 minutes and uploaded to Illinois Media Space at https://mediaspace.illinois.edu.

Your video demo should address the following:

- What is the application scenario? Who are the target users? What are the objectives?
- What databases have you used? How?
- What are the widgets? Present each widget. Explain
    - how to use it,
    - how it contributes to the objectives,
- How did you implement the application?
- What are the database techniques?
- (Optional) What are extra-credit capabilities, if any?

- How does the dashboard as a whole, with all the widgets, accomplish the objectives? Demonstrate it with a complete use case.

Specifications / Requirements

Application

Your application should serve some target users in some scenarios for accomplishing a concrete task using the Academic World dataset. It should be:

- Useful- the target users will use it for something beneficial (**R4**).
- Cool- such an application does not commonly exist today (**R5**).

Backend: Databases

The backend of your application consists of the **Academic World dataset** stored in the three databases that we have built: MySQL in the MP4 Problem 1 (**R6**), MongoDB in MP3 Problem 4 (**R7**), and Neo4j in MP3 Problem 5 (**R8**). Your frontend will communicate with the databases using their respective query languages.

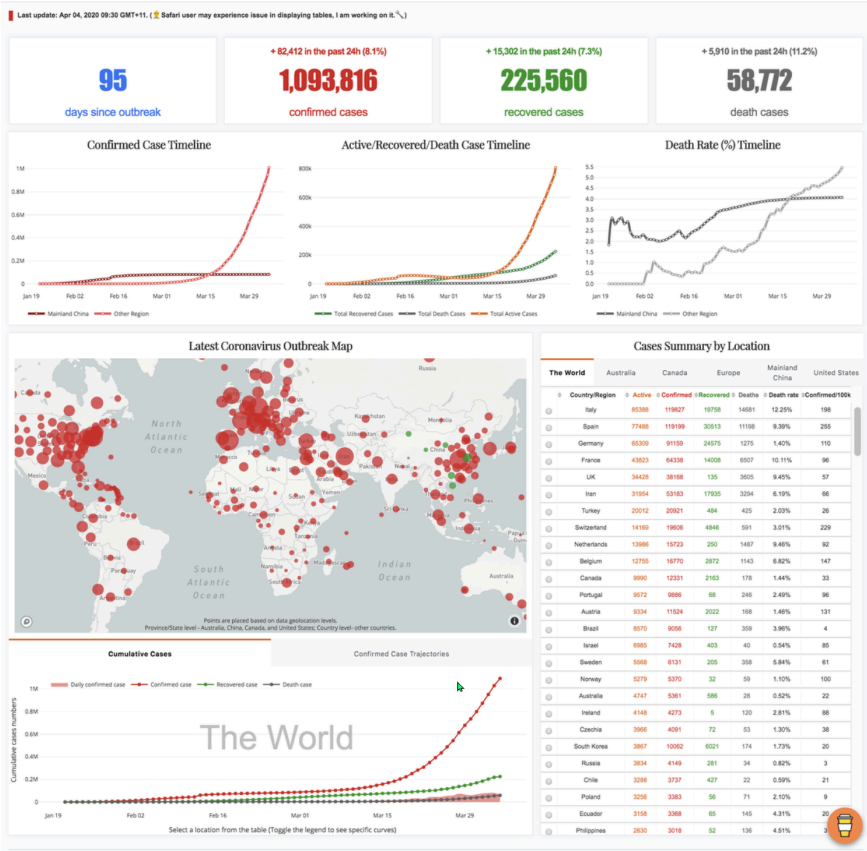
Noted:

- The data in MySQL, MongoDB and Neo4j may not be consistent because some data may be modified or removed when generating the data files and skipped when loading into the databases. The consistency of data across databases is not required.
- While you **must** use the provided dataset, you can add more data suitable for your application scenario. E.g., you may add data about users– e.g., their personal information or preferences.

Frontend: Web-based Data Dashboard

The frontend of your application is a website that presents a **data dashboard** for using the dataset in the backend databases. A data dashboard consists of a set of **widgets**– each of which is a functional unit providing an independent capability– arranged in a rectangular space to support a user to manipulate and monitor the data in your databases, much like the dashboard of a vehicle can allow a driver to control and visualize the status of the vehicle.

Below is the screenshot of an example dashboard that reports coronavirus data. In this example, there are eight widgets: Four in the first row which present summary figures, one in the second row which has three trending timeline charts and, below it, a geographic visualization, a “The World” trending, and a “Case Summary by Location”. You can see more examples here.



Your dashboard should satisfy:

- **Sufficient widgets:** It consists of at least **six** widgets, each manipulates or monitors one or more databases (**R9**).
- **Updating widgets:** It has at least **two** widgets that perform updates of the backend databases (**R10**).
- **Querying widgets:** It has at least **four** widgets that take input from users (**R11**).
- **Visual Layout:** The widgets should be arranged in a rectangular space with a layout that is visually pleasing, informative, and logical (**R12**).

Noted:

- For **R9**, if two or more widgets are triggered by one input, these widgets can be considered as different widgets, but they need to have different functionalities.
- For **R10** and **R11**, you may update or query the data added by yourself, but you should guarantee that the given **Academic World dataset** stored in the three databases each is used in **at least one** widget.
- **A widget can have multiple functions**. For example, a widget has both updating and querying functions. In this case, it can count as both the updating widget for **R10**, and the querying widget for **R11**.
- Both updating and querying widgets should **display their results** on the interface to show they really work.

Database Techniques

Use at least **three** of the following techniques (**R13, R14, R15**) in your databases.

1. Indexing
2. View
3. Constraint (Check the "*Foreign Keys and Other Constraints*" section in Alter-Table)
4. Trigger
5. Stored procedure
6. Prepared statements
7. Transaction
8. Parallel query execution (Parallel query execution in Neo4j and MongoDB)
9. Partitioning/sharding (Partitioning in MySQL and MongoDB)
10. REST API for accessing databases (REST API for MySQL and Neo4j)

Noted:

- Primary keys defined in AcademicWorld.sql **do not** count for the indexing and constraint requirements.

Implementation

Libraries/Framework

The implementation will require web development and databases. You can use **any** libraries/framework for your development.

Minimal Implementation

To minimize programming complexity so you can focus on the essence of an end-to-end database application, we suggest that you use a dashboard framework, such as Dash Plotly or Jupyter Dashboards. We have tested a few sample projects using Dash.

Using Dash, you can build your project entirely in Python with quite a minimal web programming. A simple, minimalistic implementation will consist of the following components:

- **Dashboard specification:** You will write a python file **app.py**, which uses Dash (and other libraries like Pandas, and Numpy) to declaratively create widgets and their layout.
- **(Optional) Website style:** You may write a style sheet file, say **style.css**, to specify the style of your website.
- **Database access:** You will write a few python functions organized in a few files like **mysql_utils.py**, **mongodb_utils.py**, and **neo4j_utils.py** to access the databases using their query languages via their network interfaces to support the widgets.
- **Databases:** You have already created, in your MPs, the databases MySQL, MongoDB, and Neo4j with the Academic World dataset. These databases will be accessed through their network interfaces.

Resources

- If you use Dash, this GitHub repo Awesome Dash: A curated list of awesome Dash (plotly) resources is a nice compilation of useful stuff to learn from and leverage, including tutorials, app examples, more examples, galleries, etc.

Extra-Credit Capabilities

As an extra-credit option, you can add additional capabilities – useful, cool, and non-straightforward functionality – to your applications. Such capabilities will extend your application beyond the basic specifications/requirements described above. You can add up to two additional capabilities. Each capability will be awarded up to 7.5 points (7.5%) depending on its sophistication, to the final grade of the class.

You can get early feedback for your ideas from the teaching staff. Ask your CA/TA in your personal chat room on Campuswire. Just provide a description of your idea: 1) What is it? 2) Why is it useful/cool? 3) How is it non-straightforward? How would you build it?

The following are some examples:

- **Multi-database querying:** Support a function that requires querying more than one database, i.e., using the result from querying MySQL to query Neo4j.
- **Data expansion:** Add more datasets to enable supporting interesting tasks. E.g., adding UIUC course data to allow finding papers useful for a course.
- **External data sourcing:** Query external services to bring in more data to support interesting tasks. E.g., searching Google Scholar to find new papers of a researcher.
- **Data cleaning:** Support a dashboard to allow users to interactively clean the Academic World dataset. E.g., a widget to find errors in data; a widget to perform batch data correction.

Extra Credit Rubric: Points will be awarded based on the following criteria, as best estimated and best determined by the staff. These points will be awarded directly to the final grades.

1. **Interestingness:** Is the capability useful and cool? This will be subjectively assessed by the staff.
2. **Effort:** How much work does it take by one proficient programmer? This will be estimated by the grading staff.

	0 - 10 hours	**10 - 20 hours**	**20 - 30 hours**	**30 - 40 hours**	**40 - 50 hours**
Exceptional	1.5	3	4.5	6	7.5
Good Enough	1	2	3	4	5
Poor	0	0	0	0	0

Suggested Approaches

While this project has only a final submission but no intermediate milestones, we suggest the following approach and steps.

1. Find a partner to form a team.
2. Read this document carefully. Check out our CAs' sample projects.
3. Decide the objectives of your application overall. Identify the target users, scenarios, and tasks to support. Give your application a nice name!
4. Design the dashboard. What widgets? What does each do? How do they together accomplish the overall application? How do they satisfy our requirements?
5. Learn to program with Dash. Go through its tutorial. Start from installation. Do a Hello World.
6. Learn how to use Python to access databases. E.g.,
    a. Using MySQL from Python
    b. Using MongoDB from Python
    c. Using Neo4j from Python
7. Program database access functions/classes, i.e., **mysql_utils.py**, **mongodb_utils.py**, **neo4j_utils.py**.
8. Program **app.py** – start with a simple layout, and add one widget at a time.
9. Investigate what database techniques are useful for your application and develop them.
10. Configure/adjust the overall layout. Create a style sheet if necessary.
11. Test and polish.
12. Create a video demo.
13. Upload code. Write README.md.

Grading Rubric

Requirement	**Weight**	**Criteria**
R1	10	Codes are submitted in Repo.
R2	10	Report/README is included in Repo.
R3	10	Valid video is provided in Report/README.
R4	10	Requirement is satisfied.
R5	10	Requirement is satisfied.
R6	10	Requirement is satisfied.
R7	10	Requirement is satisfied.

R8	10	Requirement is satisfied.
R9	10	Requirement is satisfied.
R10	10	Requirement is satisfied.
R11	10	Requirement is satisfied.
R12	10	Requirement is satisfied.
R13	10	Requirement is satisfied.
R14	10	Requirement is satisfied.
R15	10	Requirement is satisfied.

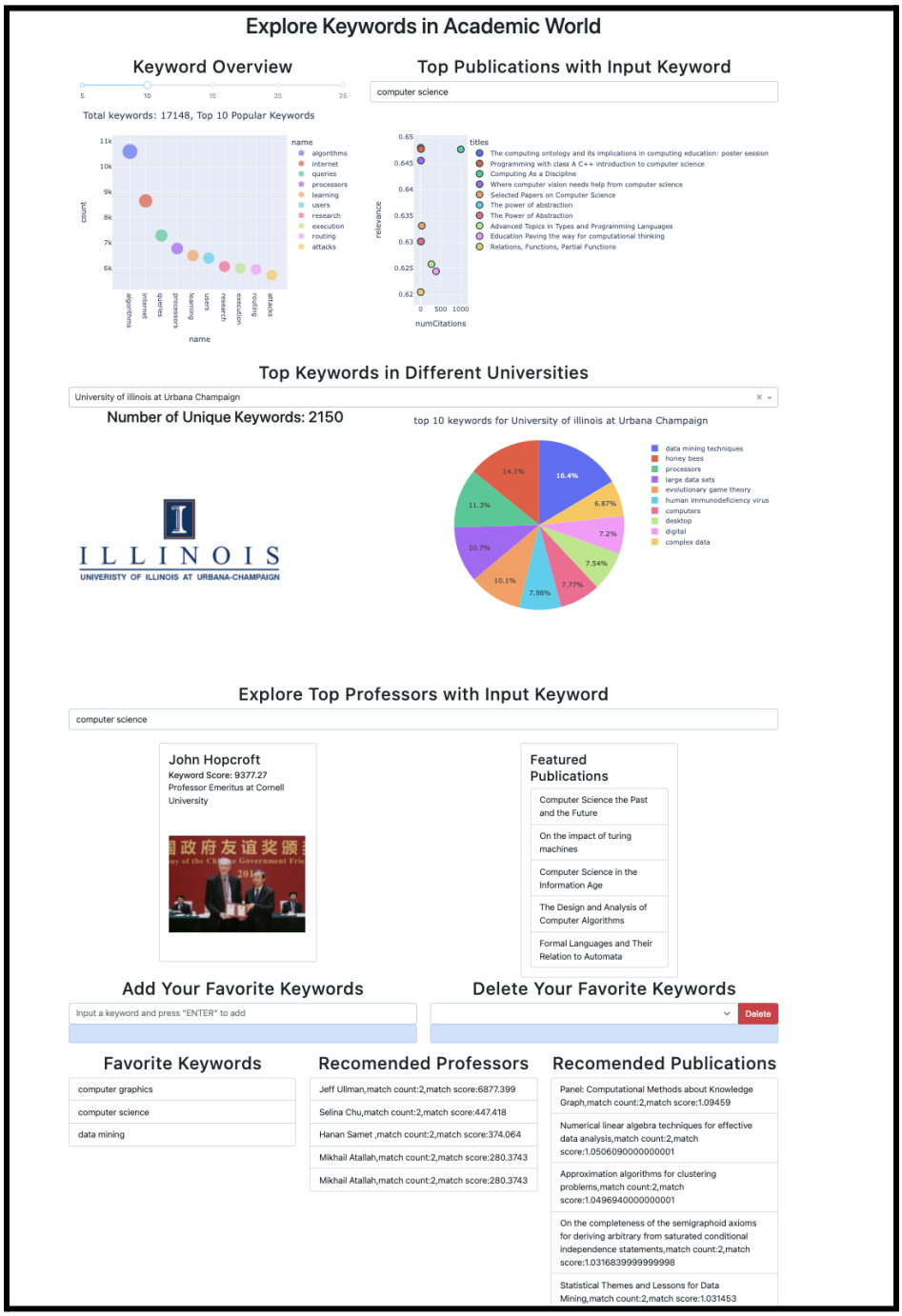
Extra-Credit Capability	**Weight**	**Criteria**
A1	0-7.5%	useful, cool, and non-straightforward
A2	0-7.5%	useful, cool, and non-straightforward

Sample Projects

The following are some example projects created by some teaching staff. They are meant to show how a project would look like. Use your imagination and creativity to build your own!

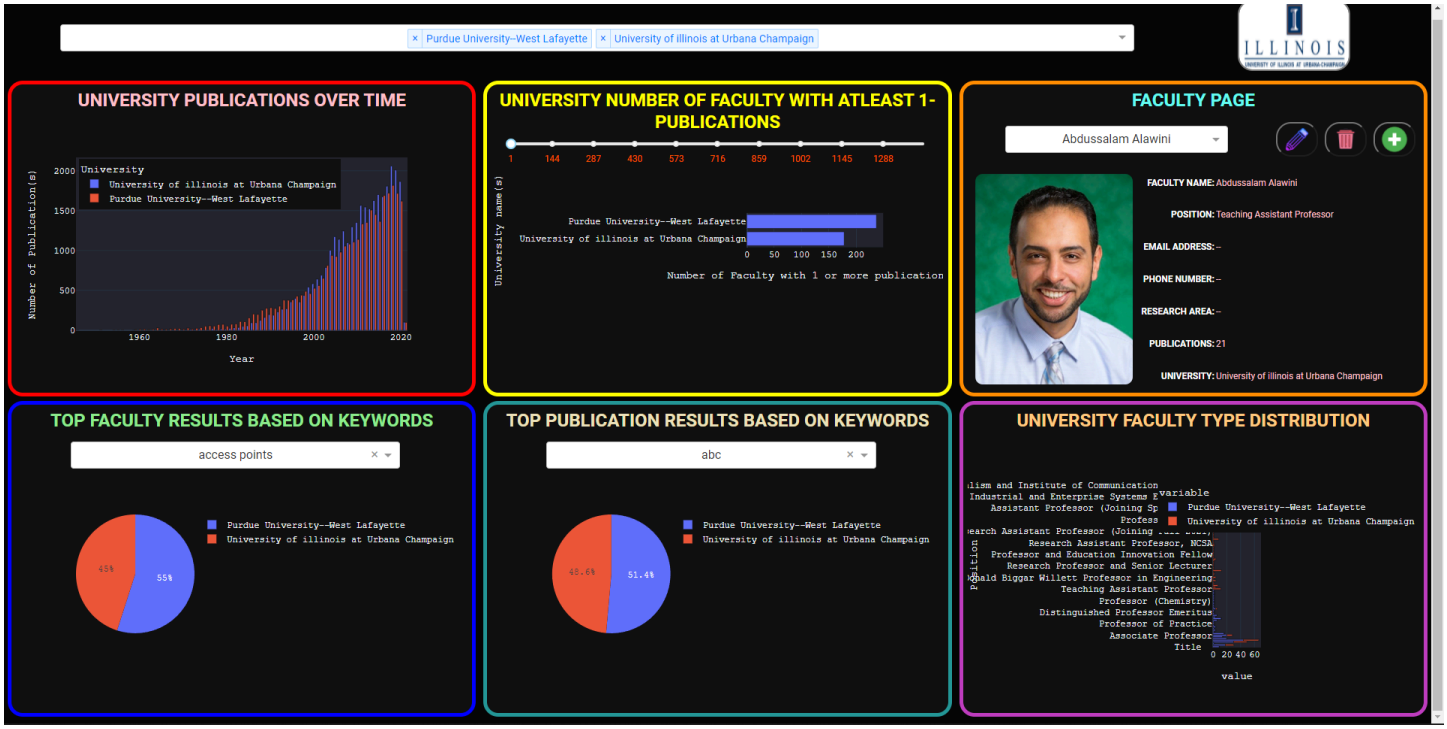
Please note: These extra projects contain only the basic application framework and do not include the required “database techniques”.

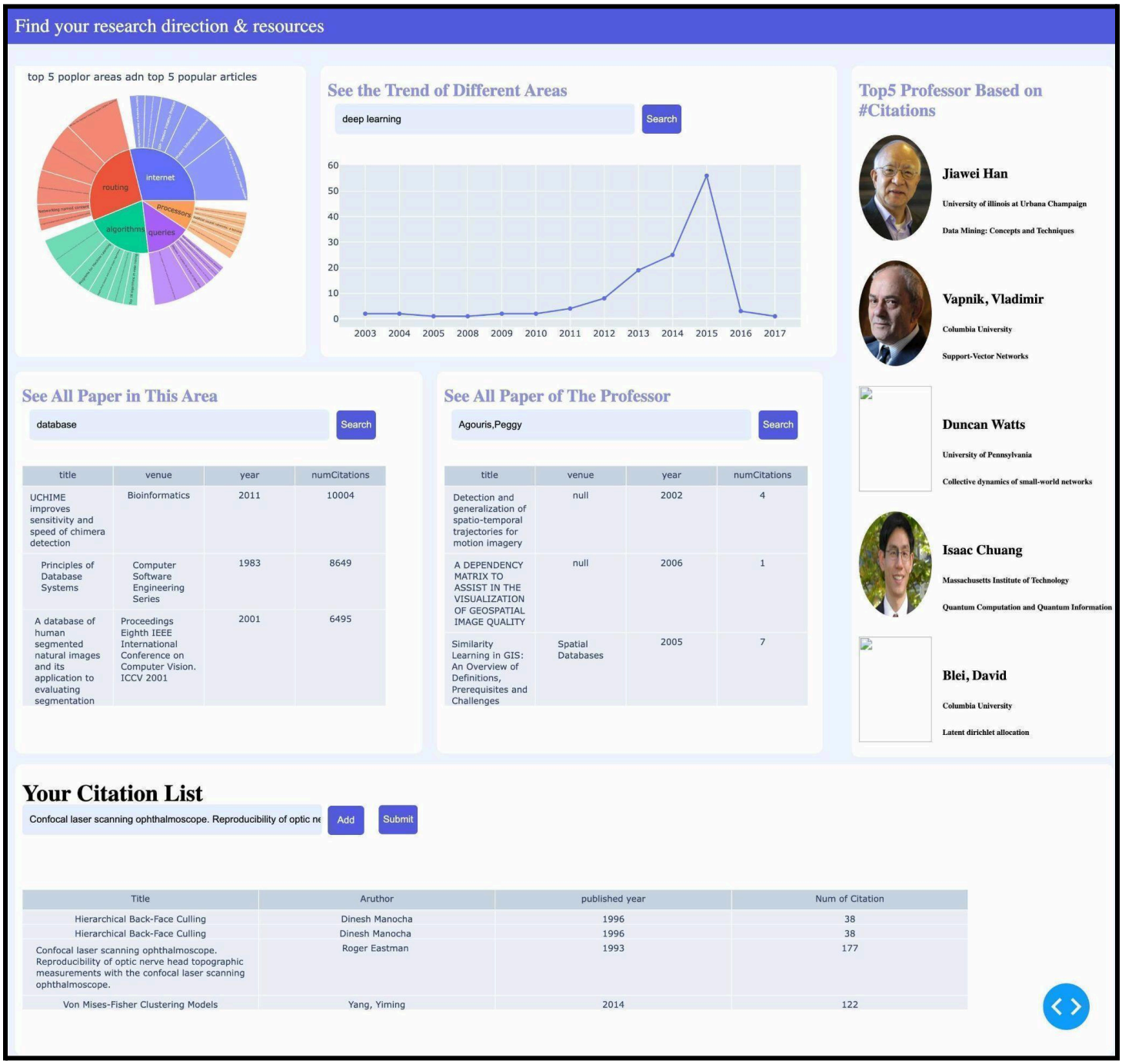
1. **Exploring Keywords in the Academic World** by Kevin Yu



2. Comparing Universities by Pratik Patel

video demo: https://youtu.be/dIofJ7RpzC



3. Finding Your Research Direction and Resources by Peiran Wang

video demo: https://youtu.be/tkZ9lEiSRfQ